1) Rotar un array hacia la derecha k posiciones
- Descripción: Dado un array de números y un entero k, gira los elementos hacia la derecha k posiciones de manera que se mantenga el orden.

```
// Ejemplo:
// let a = [1,2,3,4,5]; rotateRightInPlace(a, 2); // [4,5,1,2,3]
```

2) Subarray con suma máxima.
- Descripción: Encontrar la suma máxima de cualquier subarray contiguo y devolver la suma y el intervalo que define el subarray.

```
// Ejemplo:
// maxSubarraySum([-2,1,-3,4,-1,2,1,-5,4]) -> { maxSum: 6, start: 3, end: 6 }
```

3) Eliminar duplicados en un array ordenado manteniendo el orden.
- Descripción: en un array ordenado, elimina los duplicados y devuelve la longitud que tiene el array generado.

```
// Ejemplo:
// let a = [1,1,2,2,3]; let len = removeDuplicatesSorted(a); // a[0..len-1] = [1,2,3]
```

4) Colocar pares antes de impares manteniendo el orden.
- Descripción: coloca los %2 == 0 al principio manteniendo el orden de los elementos de cada grupo.

```
// Ejemplo:
// let a = [3,1,2,4,5]; stablePartitionEvenBeforeOdd(a); // [2,4,3,1,5]
```

5) Subarray con producto máximo
- Descripción: encontrar la estructura de subarray que tiene el producto máximo; gestiona ceros y negativos de forma iterativa.

```
// Ejemplo:
// maxSubarrayProduct([2,3,-2,4]) -> 24
```

6) Producto de los elementos sin división (incluir todos menos el propio elemento)
- Descripción: Dado un array de números, obtener un nuevo array donde cada elemento es el producto de todos los otros elementos.
Recomendación: Hacedlo en una pasada iterativa con ayuda de prefijos y sufijos.

```
// Ejemplo:
// console.log(productsExceptSelf([1, 2, 3, 4])); // [24, 12, 8, 6]
```

IES MARIA ENRIQUEZ

7) Invertir con recursión (en el mismo array o crea una copia que este invertida)
- Descripción: Invertir un array recursivamente (sin utilizar métodos de array como reverse). Devuelve una nueva copia invertida.

```
// Ejemplo:
// console.log(reverseArrayRecursive([1, 2, 3, 4])); // [4, 3, 2, 1]
```

IES MARIA ENRIQUEZ